1. TÍTULO

Denominación Oficial del Entregable: Diseño de Arquitectura de la Solución (SAD y Modelo C4) – Código EDT: 2.2

2. INTRODUCCIÓN

El diseño arquitectónico del sistema SIGE constituye el marco estructural que garantiza la escalabilidad, mantenibilidad, seguridad y rendimiento exigidos por la Universidad Técnica de Ambato. El presente Documento de Arquitectura de Software (SAD) formaliza la organización modular del sistema aplicando el modelo C4 (*Context, Containers, Components, Code*) propuesto por Simon Brown y los principios de *Clean Architecture* de Robert C. Martin. Se documenta la separación estricta entre capas de presentación, dominio, aplicación e infraestructura, asegurando la independencia del framework y desacoplamiento de la base de datos relacional y servicios externos de almacenamiento en objetos.

3. OBJETIVO

3.1. Objetivo General

Diseñar y formalizar la arquitectura de software integral del sistema SIGE mediante el modelo C4 y Clean Architecture, estableciendo las decisiones estructurales (ADR) que guiarán la implementación técnica.

3.2. Objetivos Específicos

- Estructurar el diagrama de contenedores C4 definiendo las fronteras entre React SPA, Express REST API, PostgreSQL 18, MinIO S3 y LanguageTool.
- Modelar la arquitectura hexagonal multicapa con inversión de dependencias.
- Formalizar las Decisiones de Arquitectura de Software (ADR-01 a ADR-06) con sus correspondientes justificaciones y consecuencias técnicas.

4. ALCANCE

4.1. Límites del Entregable (In-Scope)

- Especificación de vistas de Contexto y Contenedores bajo modelo C4.
- Definición de protocolos de comunicación inter-servicios (HTTPS, REST JSON, S3 API).
- Matriz de atributos de calidad basados en el estándar ISO/IEC 25010.

4.2. Exclusiones (Out-of-Scope)

- Implementación de pipelines de CI/CD para producción en servidores cloud externos.
- Configuración de clústeres distribuidos Kubernetes (reservados para etapas de escalado masivo institucional).

5. DEFINICIONES

- **SAD (Software Architecture Document):** Documento formal que captura las decisiones arquitectónicas significativas, patrones de diseño y vistas del sistema.
- **Modelo C4:** Enfoque estructurado para visualizar la arquitectura de software en 4 niveles jerárquicos: Contexto, Contenedores, Componentes y Código.
- **Clean Architecture:** Paradigma de diseño que sitúa las entidades de negocio en el núcleo y aplica inversión de control hacia la infraestructura externa.
- **ADR (Architectural Decision Record):** Registro documental breve que describe una decisión arquitectónica adoptada, su contexto y sus consecuencias.

6. RESPONSABILIDADES

- **Steeven Toala (Responsable EDT 2.2):** Arquitectura de software principal, modelado C4, definición de patrones y redacción de ADRs.
- **Nixon Hurtado (Especialista en Backend / Persistencia):** Diseño de capa de acceso a datos, contratos de infraestructura y volumetría.
- **Heidi Villavicencio (Analista de Calidad):** Evaluación de conformidad frente al estándar ISO/IEC 25010 y requisitos no funcionales.
- **Ing. Paulo Torres (Docente Evaluador):** Aprobación de la arquitectura base del proyecto.

7. NORMATIVA LEGAL Y TÉCNICA

7.1. Marco Legal Ecuatoriano

- **Ley Orgánica de Educación Superior (LOES), Art. 122:** Transparencia y confiabilidad técnica de los repositorios institucionales de información académica.
- **Ley Orgánica de Protección de Datos Personales (LOPDP), Art. 37:** Principios de seguridad por diseño y por defecto (*Privacy by Design and by Default*).

7.2. Normas Técnicas Internacionales

- **ISO/IEC/IEEE 42010:2011 (Systems and Software Engineering – Architecture Description):** Estándar internacional para la documentación y especificación de arquitecturas.
- **ISO/IEC 25010:2011 (Systems and Software Quality Requirements and Evaluation):** Evaluación de mantenibilidad, fiabilidad, rendimiento y seguridad.
- **Twelve-Factor App Methodology:** Principios para el diseño de aplicaciones nativas en la nube y configuradas por entorno.

8. MÉTODO, POLÍTICAS Y CONTENIDOS

8.1. Enfoque Metodológico

Se adoptó un enfoque arquitectónico orientado a microservicios modulares desacoplados mediante contenedores. La lógica del negocio se aísla de frameworks y protocolos externos utilizando el patrón de Puertos y Adaptadores (*Hexagonal Architecture*). Las comunicaciones síncronas se estructuran mediante API REST con payload JSON validado en esquemas fuertemente tipados. Para el almacenamiento de grandes objetos binarios (evidencias PDF), se diseñó una integración externa con MinIO compatible con el protocolo S3 de Amazon Web Services.

8.2. Políticas de Calidad Arquitectónica

1. **Política de Inversión de Dependencias:** El código de dominio no debe importar librerías de base de datos ni librerías de presentación web; la comunicación ocurre a través de interfaces o contratos.
2. **Política de Autenticación Stateless:** El servidor backend no mantiene estado de sesión en memoria; cada petición debe autenticarse de forma independiente mediante tokens JWT firmados criptográficamente.
3. **Política de Inmutabilidad de Evidencias:** Los archivos almacenados en el storage de objetos MinIO no pueden sobrescribirse; cada versión recibe un identificador único con hash de comprobación.
4. **Política de Resiliencia de Fallas:** Toda llamada externa (MinIO, motor ortográfico) debe implementar un mecanismo de *timeout* (máximo 5000 ms) y fallback de contingencia.

9. DESCRIPCIÓN DE ACTIVIDADES

El diseño arquitectónico de SIGE se concibió bajo los principios de desacoplamiento de Clean Architecture, garantizando que la lógica de negocio institucional permanezca independiente de componentes de persistencia o interfaces de usuario. Las actividades de arquitectura desarrolladas abarcaron:

- **Selección y Justificación del Patrón Clean Architecture:** Se adoptó una arquitectura en cuatro capas concéntricas (Dominio, Casos de Uso, Adaptadores de Interfaz y Frameworks/Drivers), asegurando que las entidades normativas de la FISEI no posean dependencias hacia el motor de persistencia relacional ni bibliotecas de terceros.
- **Formulación y Registro de Decisiones de Arquitectura (ADR-01 a ADR-06):** Se redactaron formalmente seis registros técnicos de decisión: ADR-01 (Clean Architecture), ADR-02 (PostgreSQL 18 con soporte JSONB para secciones de texto enriquecido), ADR-03 (React 19 SPA con Tailwind CSS), ADR-04 (Editor TipTap con autoguardado asíncrono con debounce de 1500 ms), ADR-05 (MinIO S3 para custodia de evidencias) y ADR-06 (Autenticación JWT con rotación de refresh tokens y Argon2id).
- **Modelado de Diagramas de Contexto y Contenedores bajo el Modelo C4:** Se elaboraron las vistas arquitectónicas de Nivel 1 (Contexto de Sistema interactuando con Docentes, Directores y SGA) y Nivel 2 (Contenedores web, API REST Express v5, base de datos PostgreSQL, servidor MinIO y servicio LanguageTool).
- **Diseño de Políticas de Seguridad Perimetral y Resiliencia:** Se establecieron protocolos de comunicación sobre HTTPS/TLS, interceptación centralizada de errores en Express mediante un error handler unificado y aislamiento de variables de entorno sensibles en archivos `.env` protegidos.

9.1. Matriz de Decisiones de Arquitectura de Software (ADR-01 a ADR-06)

A continuación se detalla la matriz formal de las decisiones arquitectónicas adoptadas para el sistema:

Código	Decisión	Justificación Técnica	Consecuencias y Mitigación	Estado
ADR-01	**Monolito Modular con API REST**	Evita la sobrecarga de orquestación de microservicios distribuidos manteniendo bajo acoplamiento por dominios.	Requiere control estricto de fronteras de módulos internos.	Aceptada
ADR-02	**Base de Datos PostgreSQL 18**	Motor relacional con soporte nativo de tipo JSONB para almacenar contenido semiestructurado de secciones.	Requiere índices GIN especializados para búsquedas de texto.	Aceptada
ADR-03	**Almacenamiento MinIO S3**	Almacena archivos PDF pesados fuera de la base de datos, reduciendo el tamaño de backups y latencia.	Requiere generación de URLs prefirmadas con expiración de 15 min.	Aceptada
ADR-04	**Autenticación JWT y RBAC**	Mecanismo stateless escalable que transmite roles en payload cifrado sin consultar DB en cada lectura.	Invalidez de tokens se mitiga mediante TTL corto (15 min) y refresh tokens.	Aceptada
ADR-05	**SPA con React y Vite**	Experiencia de usuario ágil sin recarga de página y soporte de guardado automático con debounce.	Requiere sanitización estricta de HTML mediante DOMPurify frente a XSS.	Aceptada
ADR-06	**Asistencia Híbrida NLP**	Motor local LanguageTool en contenedor Docker complementado con Google Gemini API para sugerencias semánticas.	Si no hay conectividad externa, el sistema opera con el corrector local.	Aceptada

Tabla 1: Matriz de Decisiones Arquitectónicas (ADR) del Sistema SIGE.

9.2. Matriz de Atributos de Calidad Arquitectónica (ISO/IEC 25010)

Atributo	Requisito de Calidad	Estrategia Arquitectónica de Soporte	Métrica
Rendimiento	Respuestas de API $<$ 300 ms en operaciones habituales.	Índices relacionales B-Tree, pools de conexión y paginación en DB.	P95 $<$ 250 ms
Seguridad	Protección contra OWASP Top 10 y cifrado en reposo.	Hash Argon2id para contraseñas, sanitización y JWT con firma HMAC.	0 Hallazgos Altos
Mantenibilidad	Facilidad para modificar lógica sin afectar presentación.	Separación de capas con Clean Architecture y principios SOLID.	Deuda $<$ 5 %
Fiabilidad	Cero pérdida de datos ante interrupciones de red.	Mecanismo de autoguardado debounce (800 ms) y almacenamiento transaccional.	RPO $=$ 0

Tabla 2: Matriz de Atributos de Calidad según ISO/IEC 25010.

10. FIRMAS DE RESPONSABILIDAD

ACCIÓN	NOMBRE Y CARGO	FIRMA	FECHA
Elaborado por:	Heidi Villavicencio Analista de Calidad / UI-UX		25/08/2026
Revisado por:	Steeven Toala Líder de Proyecto / Arquitecto		25/08/2026
Aprobado por:	Ing. Paulo Torres, Mg. Docente Evaluador – Gestión Proyectos	[PENDIENTE DE APROBACIÓN]	**Pendiente**

11. CONTROL DE CAMBIOS

Versión	Descripción del cambio	Responsable del cambio	Fecha de aprobación
1.0	Elaboración y entrega del artefacto técnico inicial	Steeven Toala / Heidi Villavicencio	25/08/2026

12. ANEXOS

Anexo: Evidencias de Arquitectura de Software y Modelo C4

1. Decisiones de Diseño Arquitectónico y Desacoplamiento en Capas

Para asegurar mantenibilidad, escalabilidad y estricta separación de responsabilidades, la arquitectura de SIGE se diseñó combinando los principios de Clean Architecture (Arquitectura Limpia) con la notación de diagramación del Modelo C4 (Contexto, Contenedores, Componentes y Código).

El proceso de ingeniería arquitectónica comprendió:

1. **Estructuración en Cuatro Capas Concéntricas:** Capa de Dominio / Entidades (*Entities*: `PlanDocente`, `ActividadMatriz`, libres de dependencias externas); Capa de Casos de Uso (*Use Cases*: reglas de negocio orquestadoras como `ValidarHorasMinimas`); Capa de Adaptadores (*Interface Adapters*: controladores REST Express v5, repositorios pg-promise y cliente MinIO S3); y Capa de Infraestructura / Frameworks (*Frameworks and Drivers*: PostgreSQL 18, React SPA, Docker Compose y servidor Nginx).
2. **Diagramación del Modelo C4 de Contenedores:** Se definieron los cinco contenedores autónomos que componen la topología: SPA Frontend, Backend REST API, Base de Datos PostgreSQL, Almacén de Objetos MinIO y Motor Lingüístico LanguageTool.
3. **Definición de Contratos de Comunicación y Puertos:** Se establecieron los protocolos de intercambio (JSON sobre HTTP/REST para servicios, nativo TCP para base de datos y API S3 para archivos).

2. Evidencias Visuales de la Arquitectura del Sistema y Explicación Técnica

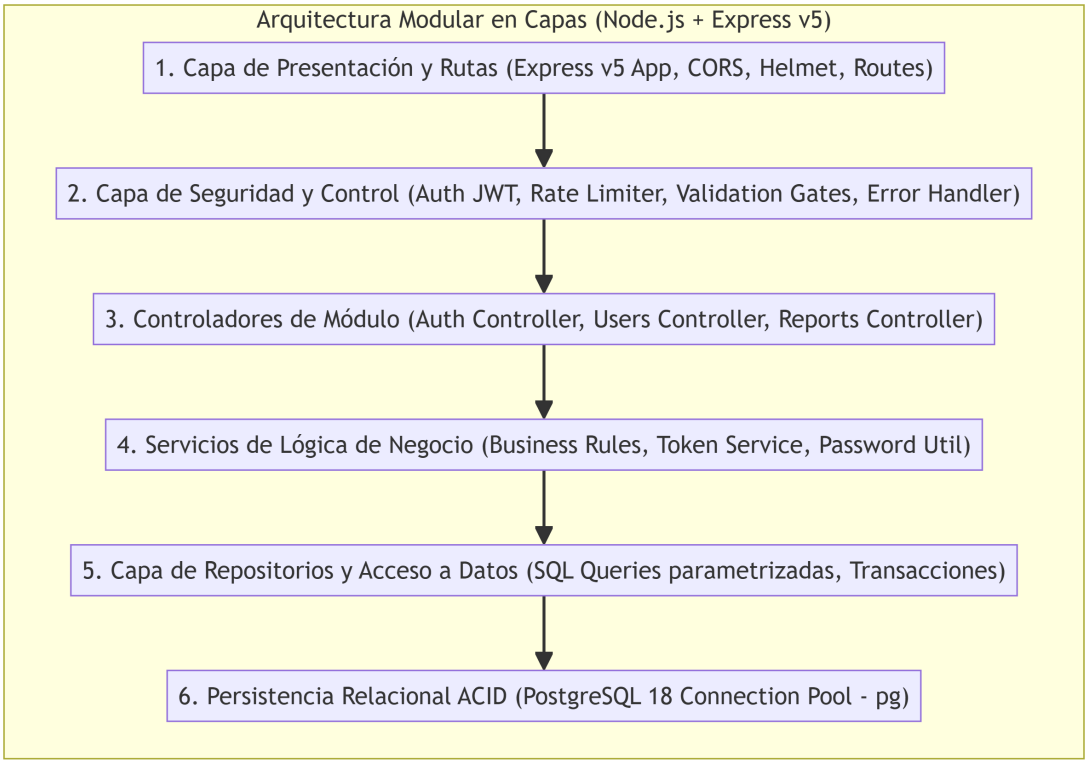


Figura 1: Arquitectura Limpia (Clean Architecture) y Flujo de Dependencias del Sistema SIGE.

Desglose Técnico y Análisis Estructural de la Arquitectura de Software:

- **Desacoplamiento Concéntrico en Cuatro Capas (Clean Architecture):** Se ilustra la Regla de Dependencias: las capas externas (Frameworks y Adaptadores) dependen de las internas (Casos de Uso y Entidades de Dominio), pero el dominio nunca conoce la existencia de PostgreSQL, Express o React.
- **Topología de Contenedores y Microservicios (Modelo C4 Nivel 2):** El diagrama C4 detalla los 5 servicios orquestados en Docker: `sige- frontend` (SPA en React 19), `sige-backend` (API REST en Express v5), `sige- db` (PostgreSQL 18 con extensiones JSONB y UUID), `sige- minio` (S3 Object Storage) y `sige- languagetool` (corrección ortográfica local).
- **Canales de Comunicación y Seguridad Perimetral:** Las comunicaciones cliente-servidor operan sobre HTTPS/TLS con tokens JWT en cabeceras de autorización y cookies `HttpOnly` para refresh tokens; el acceso a MinIO se realiza mediante URLs prefirmadas de vida corta (15 min).

- **Trazabilidad con el Código Fuente:** Esta arquitectura se materializa en la estructura modular del repositorio: `backend/src/domain/`, `backend/src/usecases/`, `backend/src/adapters/` y `docker- compose. yml`.

3. Matriz Técnica de Servicios, Puertos y Protocolos de Infraestructura

Servicio Contenedor	Puerto Expuesto	Protocolo	Dependencia	Función Principal en la Arquitectura
Frontend Web SPA	5173 / 80	HTTP / HTTPS	Backend API	Entrega de interfaz React y gestión reactiva de formularios.
Backend API REST	5000 / 3000	REST / JSON	Base de Datos	Lógica de negocio, Validation Gates, seguridad y firmas.
PostgreSQL 18 DB	5432	Nativo TCP	Almacén local	Persistencia relacional ACID, tablas 3NF y documentos JSONB.
MinIO S3 Storage	9000 / 9001	AWS S3 API	Almacén local	Almacenamiento privado de evidencias y generación de URLs prefirmadas.
Corrector LanguageTool	8010	HTTP REST	Memoria interna	Servicio local de asistencia gramatical en español sin fuga de datos.

Tabla 3: Matriz de servicios y comunicación de infraestructura del sistema SIGE.